
Palette Diffusion Curriculum Reinforcement Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

Curriculum Reinforcement Learning (CRL) seeks to improve agent learning efficiency by arranging tasks in a progressively challenging sequence. Current CRL approaches often encounter difficulties in steering agents toward intended goals, especially when domain-specific knowledge is unavailable. To overcome this limitation, we propose PALDIFFCURL (Palette Diffusion Curriculum Reinforcement Learning), an innovative CRL approach that utilizes a mask conditional image-based pre-trained diffusion model to craft curriculum objectives, takes the agent trajectory as a heatmap, and outputs curriculum goal candidates. To select an optimal curriculum from the curriculum goal candidates, we use different strategies, such as averaging, Q -function, and a trainable reward function mechanism. This integrated approach enables the formulation of goals that are challenging yet feasible, allowing agents to acquire skills effectively without requiring domain expertise. We evaluate PALDIFFCURL’s performance through comprehensive experiments in three distinct maze settings, where it either exceeds or equals the performance of ten prominent CRL algorithms from prior works.

1 Introduction

Reinforcement Learning (RL) enables agents to learn optimal actions through trial-and-error interactions with the environment, guided by a reward signal to adjust their strategies to maximize cumulative rewards. A key challenge in RL is efficiently exploring large state spaces, which becomes increasingly costly as their size grows [1]. Curriculum Reinforcement Learning (CRL) provides the agent with instructive sequences that progressively lead it toward the target objective. Several strategies exist for curriculum goal generation. Uncertainty-based methods leverage exploration guidance but can be less effective in large goal spaces [2, 3, 4]. Other approaches involve interpolating between source and target task distributions [5, 6, 7, 8] or minimizing the distance (e.g., Euclidean) between the distribution of achieved states and a desired goal distribution, though distance metrics may be suboptimal in certain environments, such as Maze [9, 10]. Generative Adversarial Networks (GANs) have also been employed for CRL, although they usually depend on arbitrary thresholds [11].

The present work starts from the hypothesis that existing CRL methods may provide a valuable source of knowledge, currently untapped, that could be used to generate curriculum goals more effectively. To this aim, we propose an innovative approach to curriculum goal generation by repurposing Palette [12]—a diffusion model originally developed for image colorization, inpainting, uncropping, and JPEG restoration—for curriculum generation. Specifically, we represent both the agent’s trajectory and the curriculum goal as heatmap images, with a marker indicating the goal location. We use a state-of-the-art CRL algorithm, OUTPACE [4], for building a dataset of such trajectories. By learning from trajectory-goal pairs, the diffusion model implicitly captures the underlying environment specificities and generates goals that are both achievable from the current trajectory and progressively challenging, without relying on explicit domain knowledge or handcrafted heuristics.

Our method adapts the powerful image-to-image translation capabilities of diffusion models to curriculum learning through two distinct phases:

40 **1. Training Phase:** We train Palette [12] on an image-based dataset of agent-trajectory heatmaps,
 41 each annotated with the curriculum goals generated in that episode. We construct this dataset using
 42 OUTPACE [4], which is the best-performing method across the set of our ten baselines (Section 4.1).
 43 During training, a binary mask highlights the region to be reconstructed (Section 4.2), enabling the
 44 diffusion model to learn to inpaint that area conditioned on the visible context.

45 **2. Inference Phase:** At inference time (Section 4.3), the model receives an agent trajectory image as
 46 input, applies a mask to specify the inpainting region, and the Palette diffusion model generates a set
 47 of curriculum goal candidates. These candidates are then evaluated using various selection strategies
 48 (Section 4.4)—namely, averaging, Q -function, and a trainable reward function—to choose a goal that is
 49 both achievable yet sufficiently challenging.

50 **2 Related Work**

51 **Palette.** Palette [12] is a versatile framework for image-to-image translation using conditional
 52 diffusion models. The framework is evaluated on four key tasks: colorization, where it adds color to
 53 grayscale images; inpainting, where it fills in missing regions; uncropping, where it extends image
 54 boundaries; and JPEG restoration, where it recovers quality from compressed images and outperforms
 55 GAN and regression baselines without task-specific tuning.

56 **Curriculum Reinforcement Learning.** CRL frameworks systematically design sequences of learn-
 57 ing experiences to enhance agent performance and training efficiency. These methods construct
 58 intermediate objectives that guide agents toward target goals, demonstrating particular utility in
 59 robotics manipulation tasks [13, 14]. Hindsight Experience Replay (HER) can be considered as an
 60 implicit curriculum approach which achieves a curriculum effect without direct manipulation of the
 61 task sequence and addresses sparse reward challenges by leveraging hindsight goals, where achieved
 62 states are treated as pseudo-goals to substitute for desired objectives [?]. However, its effectiveness
 63 diminishes when target goals are distant from initial states. Building upon HER, Hindsight Goal
 64 Generation (HGG) [9] improves this limitation by combining value function maximization with
 65 Wasserstein distance minimization between achieved and desired goal distributions.

66 Other methods, such as CURROT [15] and GRADIENT [16], employ optimal transport principles
 67 for curriculum design. CURROT reformulates CRL as a constrained optimization problem using
 68 Wasserstein distance to quantify distributional divergence, while GRADIENT introduces task-specific
 69 contextual metrics and accommodates non-parametric distributions in diverse settings. Self-Paced
 70 Reinforcement Learning (SPRL) [5] adapts principles from self-paced learning to RL. It generates
 71 curricula by considering two main factors: the agent’s value function, which serves as an indicator
 72 of task complexity, and the KL divergence from a target distribution of tasks, ensuring that the
 73 curriculum progresses towards desired objectives.

74 GoalGAN [11], instead, utilizes GANs [17] to generate intermediate goals without explicit target
 75 distribution alignment. A discriminator evaluates goal difficulty relative to the current policy, while a
 76 generator proposes regions via indicator reward functions. Policies are conditioned on both state and
 77 goal, akin to universal value function approximators [18].

78 PLR [2] prioritizes experiences with high Temporal-Difference (TD) error-based learning potential,
 79 creating a self-paced curriculum. VSD [19] selects goals by estimating epistemic uncertainty in
 80 value function predictions, favoring goals with balanced confidence. ACL [20] maximizes learning
 81 progress through metrics like prediction accuracy improvements and network complexity growth.

82 ALP-GMM [8] fits Gaussian mixtures using Absolute Learning Progress (ALP) scores, derived from
 83 reward differences across episodes. Goals are sampled to maximize student progress modeled by
 84 these mixtures. OUTPACE [4] employs Adversarial Intrinsic Motivation (AIM) [21] to minimize
 85 the Wasserstein distance between state visitation and goal distributions. It generates curricula
 86 via Conditional Normalized Maximum Likelihood (CNML) to classify goal-associated states and
 87 prioritizes uncertain, temporally distant objectives using meta-learning [22] and Wasserstein-based
 88 temporal approximations.

89 **3 Background**

90 We now introduce the background concepts on multi-goal RL, Soft Actor-Critic (SAC), Wasserstein
 91 distance, AIM, and diffusion models, which are the main elements of our approach.

92 3.1 Multi-Goal Reinforcement Learning

93 We formulate the multi-goal RL problem as a goal-conditioned Markov Decision Process (MDP),
 94 defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{G}, \mathcal{T}, R, \rho_0, \gamma_r \rangle$. Here, $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space,
 95 and $\mathcal{G}$ is the space of possible goals. The transition dynamics $\mathcal{T}(s'|s, a)$ gives the probability of
 96 transitioning to state s' after taking action a in state s . The initial state s_0 and the desired goal g
 97 for an episode are sampled from a joint distribution $\rho_0(s_0, g)$. The reward function $R(s, a, g, s')$
 98 provides feedback, and $\gamma_r \in [0, 1)$ is a discount factor. The agent’s objective is to learn a goal-
 99 conditioned policy $\pi(a|s, g)$ that maximizes the expected discounted cumulative reward: $J(\pi) =$
 100 $\mathbb{E}_{s_0, g \sim \rho_0, a_t \sim \pi(\cdot|s_t, g), s_{t+1} \sim \mathcal{T}(\cdot|s_t, a_t)} [\sum_{t=0}^{\infty} \gamma_r^t R(s_t, a_t, g, s_{t+1})]$. In this work, we utilize the AIM
 101 [21] reward function r_φ , detailed in Section 3.3, to provide denser learning signals. Building upon
 102 this multi-goal RL framework, we further enhance the agent’s learning efficiency by incorporating
 103 curriculum goals. CRL structures the training process by presenting tasks in a sequence of increasing
 104 difficulty, allowing the agent to master simpler tasks before progressing to more complex ones.

105 3.2 Soft Actor-Critic

106 SAC [23] is an off-policy algorithm for RL that optimizes a policy to maximize both expected cumula-
 107 tive rewards and policy entropy: $J(\pi) = \sum_{t=0}^T \mathbb{E}_{s_t \sim \mathcal{B}} r + \alpha_e \mathcal{H}(\pi(\cdot | s_t, g))$. Within this framework,
 108 $\pi_\psi(a | s, g) : \mathcal{S} \mapsto \mathcal{P}(\mathcal{A})$ establishes a mapping between states and probability distributions over ac-
 109 tions, with parameters ψ . The value of state-action pairs is captured by $Q_\phi(s, g, a) : \mathcal{S} \times \mathcal{G} \times \mathcal{A} \rightarrow \mathbb{R}$,
 110 parameterized by ϕ . The optimization of policy parameters proceeds through minimization of the
 111 expected Kullback-Leibler divergence, while the Q -value parameters are refined by minimizing the
 112 soft Bellman residual, expressed through the following loss functions:

$$\mathcal{L}_\pi = \mathbb{E}_{(s, g) \sim \mathcal{B}} [\mathbb{E}_{a \sim \pi_\psi} [\alpha_e \log (\pi_\psi(a | s, g) - Q_\phi(s, g, a))]] \quad (1)$$

$$\mathcal{L}_Q = \mathbb{E}_{(s, a, r, s', g) \sim \mathcal{B}} [Q_\phi(s, g, a) - (r + \gamma_r \mathbb{E}_{s' \sim \mathcal{B}} [V_\phi(s', g)])^2] \quad (2)$$

113 where $V_\phi(s, g) = \mathbb{E}_{a \sim \pi} [Q_\phi(s, g, a) - \log \pi_\psi(a | s, g)]$, $\mathcal{B}$ is the replay buffer, and α_e is the temper-
 114 ature parameter that controls the stochasticity of the optimal policy.

115 3.3 Wasserstein Distance and Adversarial Intrinsic Motivation Reward Function

116 The AIM reward function can be learned, as suggested by Durugkar et al. [21], through the mini-
 117 mization of the Wasserstein distance between the state visitation distribution ρ_π and the target goal
 118 distribution $\mathcal{G}$. The minimization of the Wasserstein distance $W_1(\rho_\pi, \mathcal{G})$, also referred to as the
 119 Kantorovich–Rubinstein distance, enables the formulation of a reward function that assesses the
 120 required effort to transition the state visitation distribution ρ_π to the desired goal distribution $\mathcal{G}$.
 121 It is expressed as follows: $W_1(\rho_\pi, \mathcal{G}) = \sup_{\|f\|_L \leq 1} [\mathbb{E}_{g \sim \mathcal{G}} [f(g)] - \mathbb{E}_{s \sim \rho_\pi} [f(s)]]$. Here, $f(s)$ is
 122 a potential function and monotonically increases along trajectories and reaches its peak value at
 123 $f(g)$. A reward function r_φ^π can be approximated by a neural network whose output increases as the
 124 states approach the desired goal $g \in \mathcal{G}$ and can be trained using the data collected by the policy π .
 125 Leveraging the estimation of the Wasserstein distance $W_1(\rho_\pi, \mathcal{G})$, the loss function for training the
 126 parameterized reward function r_φ^π is defined as follows:

$$\mathcal{L}_\varphi = \mathbb{E}_{(s, g) \sim \mathcal{B}} [f_\varphi^\pi(s) - f_\varphi^\pi(g)] + \lambda \cdot \mathbb{E}_{(s, s', g) \sim \mathcal{B}} [\max(|f_\varphi^\pi(s) - f_\varphi^\pi(s')| - 1, 0)^2] \quad (3)$$

127 where the second component of the sum is a penalty term and the coefficient λ is necessary to ensure
 128 smoothness [21]. The reward, which is the negative of the Wasserstein distance $-W_1(\rho_\pi, \mathcal{G})$, can
 129 then be calculated as follows¹:

$$r_\varphi(s, g) = f_\varphi^\pi(s) - f_\varphi^\pi(g). \quad (4)$$

130 3.4 Diffusion Models

131 Diffusion models [24] express a probability distribution $p(x_0)$ through latent variables in the form
 132 $p_\theta(x_0) := \int p_\theta(x_{0:N}) dx_{1:N}$, where $x_1, \dots, x_N$ are latent variables of the same dimensionality as

¹Note that $r_\varphi(s, g)$ is used both as the reward signal (maximized by the policy) and, when the initial state s_0 is fixed, as a measure to evaluate goal proximity during curriculum selection (minimized to find accessible goals). Appendix F elaborates on this dual interpretation.

the data $\mathbf{x}_0 \sim p(\mathbf{x}_0)$. These models are characterized by a forward and a reverse diffusion process. The forward diffusion process approximates the posterior $q(\mathbf{x}_{1:N} | \mathbf{x}_0)$ using a Markov chain that perturbs the input data by gradually adding Gaussian noise $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ in N steps with a predefined variance schedule $\beta_1, \dots, \beta_N$. This process is defined as:

$$q(\mathbf{x}_{1:N} | \mathbf{x}_0) := \prod_{k=1}^N q(\mathbf{x}_k | \mathbf{x}_{k-1}) \quad (5)$$

$$q(\mathbf{x}_k | \mathbf{x}_{k-1}) := \mathcal{N}(\mathbf{x}_k; \sqrt{1 - \beta_k} \mathbf{x}_{k-1}, \beta_k \mathbf{I}).$$

The reverse diffusion process aims to recover the original input data from the noisy (diffused) data. It learns to progressively reverse the diffusion process step by step and approximates the joint distribution $p_\theta(\mathbf{x}_{0:N})$. This process is defined as:

$$p_\theta(\mathbf{x}_{0:N}) := p(\mathbf{x}_N) \prod_{k=1}^N p_\theta(\mathbf{x}_{k-1} | \mathbf{x}_k) \quad (6)$$

$$p_\theta(\mathbf{x}_{k-1} | \mathbf{x}_k) := \mathcal{N}(\mathbf{x}_{k-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_k, k), \boldsymbol{\Sigma}_\theta(\mathbf{x}_k, k))$$

where $p(\mathbf{x}_N) = \mathcal{N}(\mathbf{x}_N; \mathbf{0}, \mathbf{I})$. The optimization of the reverse diffusion process is achieved by maximizing the evidence lower bound (ELBO) $\mathbb{E}_q \left[\ln \frac{p_\theta(\mathbf{x}_{0:N})}{q(\mathbf{x}_{1:N} | \mathbf{x}_0)} \right]$ [25]. Once trained, sampling data from Gaussian noise $\mathbf{x}_N \sim p(\mathbf{x}_N)$ and running through the reverse diffusion process from $k = N$ to $k = 0$ yields an approximation of the original data distribution.

4 Methodology

In multi-goal RL, the agent is tasked with achieving multiple goals, each sampled from a goal distribution at the start of every episode. By integrating a curriculum design into multi-goal RL, we restructure the task to begin with simpler goals and progressively increase the task difficulty. To generate these curriculum goals, we utilize Palette [12], an image-to-image diffusion model, originally developed for image inpainting, which we repurpose to create curriculum goals for CRL.

As RL and diffusion processes operate on distinct temporal values, we denote the diffusion iteration index as $k \in \{0, \dots, N\}$ and the RL trajectory index as $t \in \{0, \dots, T\}$.

The curriculum goal generation process begins with training a conditional diffusion model. This training involves preparing a dataset of images where each image represents an RL episode. These images encode the agent’s trajectory as a heatmap where pixel intensity reflects the frequency or recency of state visits—and the goal as a distinct black box marking its location. During inference (i.e., during the reverse-diffusion process), the trained diffusion model generates curriculum goals for the RL agent during its roll-out phase. Specifically, during RL training, we construct the agent’s trajectory heatmap image $\mathcal{I}$ for one episode and apply a mask to occlude the trajectory part on the image and a random subset of pixels. The masked region is initialized with Gaussian noise, while unmasked pixels retain their original values (Section 4.2). The diffusion model then iteratively denoises the masked region, conditioned on the masked pixels of the original image. The final output is an image combining the curriculum goal (in the denoised region) with the agent’s trajectory heatmap.

Recall that the neural network f_θ is trained to predict ϵ given a noisy image $\mathcal{I}_N$ and $\mathcal{I}_k$. Consequently, once $\mathcal{I}_k$ is known, we estimate $\mathcal{I}_0$ by rearranging the terms in Eq. (6), yielding:

$$p_\theta(\mathcal{I}_{0:N} | \mathcal{I}_N) = p(\mathcal{I}_N) \prod_{k=1}^N p_\theta(\mathcal{I}_{k-1} | \mathcal{I}_k, \mathcal{I}_N) \quad (7)$$

where:

$$p(\mathcal{I}_N) = \prod_{i,j} \begin{cases} \delta(\mathcal{I}_{N,i,j} - \mathcal{I}_{0,i,j}) & \text{if } \mathcal{M}_{i,j} = 0 \\ \mathcal{N}(\mathcal{I}_{N,i,j}; 0, \sigma^2) & \text{if } \mathcal{M}_{i,j} = 1. \end{cases} \quad (8)$$

$\mathcal{I}_N$ is obtained by applying a binary mask to the image, so as to control which part of the image should undergo the diffusion model’s noising and denoising process:

$$\mathcal{I}_N = \mathcal{I}_0 \odot (1 - \mathcal{M}) + \eta \odot \mathcal{M} \quad (9)$$

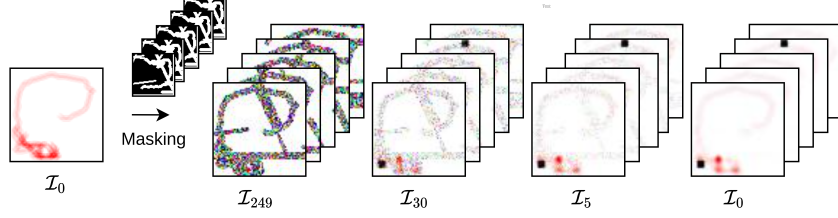


Figure 1: A masked heatmap batch is denoised using the Palette diffusion model to reconstruct the agent’s trajectory and curriculum goals.

where $\mathcal{M}$ is the binary mask, $\mathcal{I}_N$ is the noisy image in the masked region, $\eta \sim \mathcal{N}(0, \sigma^2)$ is Gaussian noise, and $\odot$ is element-wise multiplication. The reverse diffusion process, parameterized by θ , is then applied, starting from the last step N and proceeding backward to step 1:

$$\mu_{\theta}(\mathcal{I}_k, \mathcal{I}_N, \bar{\alpha}_k) = \frac{1}{\sqrt{\alpha_k}} \left(\mathcal{I}_k - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} f_{\theta}(\mathcal{I}_k, \mathcal{I}_N, \bar{\alpha}_k) \right) \quad (10)$$

$$\mathcal{I}_{k-1} | \mathcal{I}_k = \frac{\mathcal{I}_k}{\sqrt{\alpha_k}} - \frac{1 - \alpha_k}{\sqrt{\alpha_k(1 - \bar{\alpha}_k)}} f_{\theta}(\mathcal{I}_k, \mathcal{I}_N, \bar{\alpha}_k) + \sqrt{1 - \alpha_k} \epsilon \quad (11)$$

where $\epsilon \sim \mathcal{N}(0, I)$. We adopt the following simplified loss function to train the conditional model f_{θ} , which approximates the noise term ϵ :

$$\mathcal{L} = \mathbb{E}_{\mathcal{I}_t, \mathcal{I}_N, \epsilon \sim \mathcal{N}(0, I)} \|\epsilon - f_{\theta}(\mathcal{I}_k, \mathcal{I}_N, \bar{\alpha}_k)\|^2. \quad (12)$$

4.1 Data Collection

The data collection for the inpainting diffusion model involves a series of systematic steps to record, process, and visualize agent trajectories. Initially, we generated a dataset by training the OUTPACE method [4], independently for each maze environment, and recorded, for each episode, the position of the generated curriculum goal along with the agent’s state sequence $\mathcal{S} = s_0, s_1, \dots, s_T$, where s_t represents the agent’s position at timestep t and T denotes the last episode’s timestep.

To visualize the agent’s behavior, we scaled and discretized its continuous positions onto an $N \times M$ pixel grid, generating a heatmap for each episode. In the heatmap (see the input image $\mathcal{I}_0$ on the left in Figure 1 for an example), darker red shades indicate regions frequently visited by the agent during that episode, while lighter shades denote less-visited areas; unvisited regions appear white. The curriculum goal for the episode is highlighted as a black square.

4.2 Mask Configuration

Masks play a critical role in guiding the inpainting process by specifying which image regions should be reconstructed by the diffusion model. In our framework, masked regions are initialized with pure Gaussian noise while preserving original pixel values in unmasked areas. The Palette diffusion model fills the masked regions with Gaussian noise. During the reverse process, the diffusion model denoises the masked regions, conditioned on the masked pixels of the input image. For our experiments, we employ a path-based mask, illustrated above the arrow in Figure 1. This mask was designed to occlude regions corresponding to the agent’s trajectory, the goal location, and a random subset of other pixels. By incorporating both trajectory context and an element of randomness, it encourages the diffusion model to generate curriculum goals that are contextually relevant to the agent’s recent path, thereby producing meaningful exploration targets.

4.3 Diffusion Model for Curriculum Goal Generation

We employ a diffusion model to generate curriculum goals. This process involves two phases: training the model and then using it for inference during RL training. By training a diffusion model on a dataset containing agent trajectory heatmaps and their corresponding curriculum goals (generated using OUTPACE [4] in our experiments) for each episode i , we aim to capture the relationship between the agent’s trajectory and the curriculum goals. In other words, the generated dataset guides

the agent through curriculum goals, and the diffusion model is trained to learn this guidance based on the provided trajectory heatmaps. To generate curriculum goals, we use the trained diffusion model in our RL training setup (Section 4.4).

Given an agent trajectory heatmap image for an episode i , a path-based mask $\mathcal{M}$ is applied to this heatmap, occluding parts of the trajectory and random pixels, as described in Section 4.2. Differently from the diffusion model training, in this case, we do not have any goal information in the input image, and we expect the trained diffusion model to generate it for us. In this case, by only using the inference (reverse) process of the diffusion model, we feed the masked image to the model and obtain the generated agent’s trajectory and curriculum goal for the next episode $i + 1$. The masked region is initialized with Gaussian noise, while unmasked pixels retain their original values. The diffusion model then iteratively denoises the masked region, conditioned on the masked pixels of the input image. The final output is an image combining the curriculum goal (in the denoised region) with the agent’s trajectory heatmap. Examples of the diffusion process are shown in Figure 4 in Appendix A.

As can be seen from the figure, the input image is given to the diffusion model (Figure 4a). Please note that the input image has no goal information. The mask is applied to the input image, and the diffusion model is used to gradually denoising the noise in the mask region. In the final output (Figure 4l), a goal is generated and used as a curriculum goal for the next episode.

4.4 Goal Selection Strategies

Instead of using a single masked image as input to the diffusion model, we propose generating a series of path-based masked images. Given a heatmap, we apply k distinct binary masks to produce a batch of k partially masked inputs, where the number of masks, k , is a hyperparameter. Each masked heatmap serves as an independent input channel to the diffusion process. This multi-channel mechanism enables the model to independently reconstruct the masked regions for each of the k inputs, yielding a batch of k denoised heatmaps containing the goal information. This process is illustrated in Figure 1. This batched masking strategy allows us to generate multiple candidate curriculum goals for the same trajectory by applying different sets of masks. Additionally, it enhances the robustness of our algorithm by mitigating failure cases where a denoised image may contain no goal information or multiple goals. Examples of such cases are shown in Figures 5 and 6 in Appendix A. A key challenge, however, is selecting the optimal curriculum goal from the generated candidates. To address this, we propose four strategies for curriculum goal selection, namely:

1. **Averaging:** Calculates the geometric mean of the coordinates of all candidate goals $\mathcal{G}_c$ and uses this average as the curriculum goal g_c for the next episode (Algorithm 3).
2. **AIM reward:** We utilize the trainable AIM reward (Section 3.3), to select a curriculum of goal candidates that maximizes the AIM reward. The AIM reward is a trainable reward function designed to estimate the agent’s proximity to the desired goal. Specifically, it quantifies the reward associated with the agent’s current position relative to the desired goal (Algorithm 4).
3. **Q -function:** This strategy utilizes the agent’s learned state-action value function, Q , and guides the agent by selecting a curriculum goal from the generated candidates $\mathcal{G}_c$. The Q -function provides an estimate of the expected cumulative reward. For each candidate goal $g_c \in \mathcal{G}_c$, we compute a score derived from the Q -function, potentially evaluated over the agent’s recent trajectory $(s_{0:T}, a_{0:T})$, denoted as $Q(s_{0:T}, a_{0:T}, g_c)$. These scores are normalized into a probability distribution using the softmax function, and a curriculum goal g_c is then sampled according to these probabilities. This method favors goals that yield higher Q -value, thereby selecting objectives aligned with the agent’s learned policy and value estimations, which promotes efficient learning (Algorithm 5).
4. **AIM reward & Q -function:** Both the AIM reward and the Q -function can be used to select the curriculum goal for the next episode. Since the AIM reward estimates the immediate reward while the Q -function predicts the cumulative reward, they can be combined with different weighting terms to effectively select the curriculum goal (Algorithm 6).

Algorithm 1 details the PALDIFFCURL steps. Algorithms 2 to 6, along with a diagram of the whole process, are reported in Appendix B. The method initializes the off-policy algorithm, replay buffer, networks, and sets the initial curriculum goal equal to the desired goal (line 3). In each episode, the agent interacts with the environment using the current curriculum goal (lines 7-9). The agent’s trajectory is then converted to a heatmap image and passed to `PaletteCurriculumGenerator` (line 11) to produce candidate curriculum goals $\mathcal{G}_c$. From these, one goal g_c is selected using the

256 GoalSelector (line 12) for the next episode. Networks are updated using minibatches from the
 257 replay buffer (lines 14-15). Success rate is measured by test rollouts that verify if the agent reaches
 258 the desired goal within threshold κ (lines 16-22).

Algorithm 1 PALDIFFCURL process

```

1: Input: No. of episodes  $E$ , no. of steps  $T$ 
2: Select an off-policy algorithm  $\mathbb{A}$  ▷ In our case,  $\mathbb{A}$  is SAC
3: Initialize replay buffer  $\mathcal{B} \leftarrow \emptyset$ ,  $g_c \leftarrow \{g_d\}$  and networks  $Q_\phi, \pi_\psi, r_\varphi$ 
4: for episode = 0 ...  $E$  do
5:   Sample initial state  $s_0$ 
6:   for  $t = 0 \dots T$  do
7:      $a_t = \pi(s_t, g_c)$ 
8:     Execute  $a_t$ , obtain next state  $s_{t+1}$ 
9:     Store transition  $(s_t, a_t, r_t, s_{t+1}, g_c)$  in  $\mathcal{B}$ 
10:     $\mathcal{I}_0 = \text{CoordtoPixel}(s_{0:T})$  ▷ Convert the trajectory to a heatmap image
11:     $\mathcal{G}_c \leftarrow \text{PaletteCurriculumGenerator}(\mathcal{I}_0)$  ▷ Algorithm 2
12:    Find  $g_c = \text{GoalSelector}(\mathcal{G}_c)$  ▷ Algorithms 3 to 6
13:    Sample a minibatch  $b$  from replay buffer  $\mathcal{B}$ 
14:    Update  $Q$  and  $\pi$  with  $b$  to minimize  $\mathcal{L}_Q$  and  $\mathcal{L}_\pi$  in Eq. (1) and in Eq. (2)
15:    Update the AIM reward function  $r_\varphi$ 
16:     $\text{success} \leftarrow 0$  ▷ Success rate
17:    Sample a desired goal  $g_d \sim \mathcal{G}$ 
18:    for  $i = 1 \dots n_{\text{testrollout}}$  do
19:       $a_t = \pi(s_t, g_d)$ 
20:      Execute  $a_t$ , obtain next state  $s_{t+1}$  and reward  $r_t$ 
21:      if  $|\phi(s_{t+1}) - g_d| \leq \kappa$  then
22:         $\text{success} \leftarrow \text{success} + 1/n_{\text{testrollout}}$ 
23: return  $\text{success}$ 

```

Table 1: Comparison of PALDIFFCURL with CRL methods from the literature (sorted by year).

Algorithm	Curriculum method	Target dist. curriculum	Geometry-agnostic	Off-policy	External reward	Venue, year
ACL [20]	LSTM	✗	✓	✗	✗	PMLR, 2017
GoalGAN [11]	GAN	✗	✓	✗	✗	PMLR, 2018
HGG [9]	$Q, \mathcal{B}, W_2$	$\mathcal{G}^+$	✗	✓	✗	NeurIPS, 2019
ALP-GMM [8]	GMM	✗	✓	✓	✗	PMLR, 2020
VDS [19]	$Q, \mathcal{B}$	✗	✓	✓	✗	NeurIPS, 2020
SPRL [5]	SRPL	$\mathcal{G}^+$	✓	✓	✗	JMLR, 2021
PLR [2]	TD-Error	✗	✓	✗	✗	PMLR, 2021
CURROT [15]	W_2	$\mathcal{G}^+, \mathcal{U}$	✗	✓	✗	PMLR, 2022
GRADIENT [16]	W_2	$\mathcal{G}^+$	✗	✓	✓	NeurIPS, 2022
OUTPACE [4]	CNML	$\mathcal{G}^+$	✓	✓	✓	ICLR, 2023
PALDIFFCURL (Ours)	🎨 Palette Diffusion	$\mathcal{G}^+$	✓	✓	✓	

259 5 Experiments

260 To validate the effectiveness of our approach, PALDIFFCURL, we performed evaluations in three
 261 challenging maze tasks, namely PointUMaze, PointNMaze, and PointSpiralMaze, simulated using
 262 MuJoCo [26]. We benchmarked PALDIFFCURL against a comprehensive set of ten established
 263 CRL baselines: ACL [20], GoalGAN [11], HGG [9], ALP-GMM [8], VDS [19], SPRL [5], PLR
 264 [2], CURROT [15], GRADIENT [16], and OUTPACE [4]. All results below are reported over five
 265 random seeds. Table 1 compares the key conceptual differences between our method and the baseline
 266 algorithms. Details on the experimental setup are reported in Appendix D. The implementation code
 267 is publicly available at <https://anonymous.4open.science/r/DiffusionOutpace-8365>.

268 5.1 Goal Selection Mechanism

269 We first conduct an ablation study to investigate the four goal selection mechanisms discussed in
 270 Section 4.4. Results are reported in Figure 18. The performance differences observed among the four

goal selection mechanisms mainly stem from the interaction between each mechanism’s underlying strategy and the specific complexities of the maze environments. Examples of curriculum goals generated by these strategies across the different maze environments are visualized in Appendix C (Figures 8–10). Furthermore, cumulative state visitation heatmaps, illustrating how the agent explores the environment under each strategy, are provided in Appendix E (Figures 11–13).

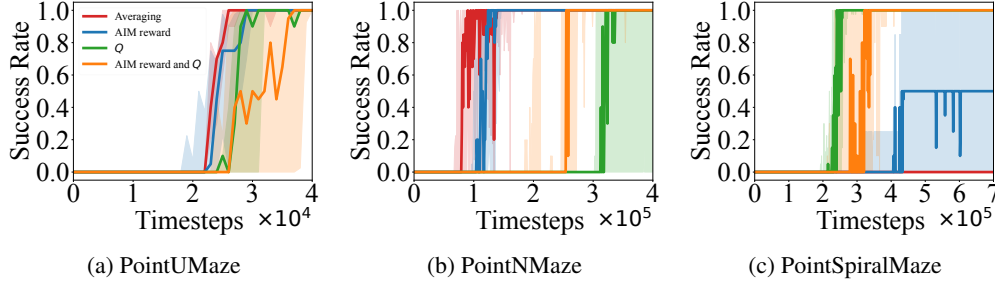


Figure 2: Performance of different goal selection strategies for PALDIFFCURL in maze tasks.

Averaging: This simple strategy (Algorithm 3) computes the geometric mean of the coordinates of all candidate goals $\mathcal{G}_c$ generated by the diffusion model. While computationally inexpensive, it lacks awareness of the environment’s geometry and constraints. As suggested by visualizations (Figures 8a, 9a and 10a), averaging can yield curriculum goals located within impassable regions (e.g., walls). In simpler environments like PointUMaze and PointNMaze (Figure 18a-b), this limitation is less detrimental. The maze structures are less convoluted, and the agent’s interaction with the environment (e.g., collisions) can often compensate for suboptimal goal placement. An averaged goal might still provide a useful directional cue, guiding exploration towards the target region. However, in the complex PointSpiralMaze (Figure 18c), which demands precise navigation, Averaging performs poorly because infeasible or poorly placed goals can easily get the agent stuck in unproductive loops.

AIM reward: This strategy (Algorithm 4) selects the candidate goal $g \in \mathcal{G}_c$ that minimizes the learned function $r_\varphi^\pi(s_0, g)$, derived from the AIM framework (Section 3.3). This function estimates the distance to reach a goal g from the initial state s_0 , without explicitly encoding the maze geometry. In simpler mazes (PointUMaze, PointNMaze, Figure 18a-b), this approach performs well, rapidly guiding the agent towards the target region by selecting goals considered most accessible (Figures 8b and 9b). Minimizing this learned proximity serves as a reasonable heuristic for progressive goal setting in less structured environments. However, its performance degrades significantly in the complex PointSpiralMaze (Figure 18c). Because the AIM proximity function lacks awareness of the intricate maze structure and long-term path feasibility, selecting goals solely based on minimizing it can lead the agent towards infeasible regions (Figure 10b).

Q-function: This strategy (Algorithm 5) leverages the agent’s learned state-action value function Q_ϕ to score candidate goals, selecting goals predicted to yield high cumulative future rewards. The effectiveness of this strategy hinges on the accuracy of the Q -function, which improves as the agent accumulates environmental experience during training. This explains the initially slower convergence observed for the Q -function selection mechanism in PointUMaze and PointNMaze (Figure 18a-b). However, as the value estimates become more reliable, this method excels at identifying goals that are both feasible and strategically advantageous for long-term reward maximization. This makes it particularly effective in complex environments like PointSpiralMaze (Figure 18c), where understanding the environmental structure and long-term consequences is crucial (Figures 8c, 9c and 10c).

AIM reward & Q-function: This combined strategy (Algorithm 6) seeks to balance the strengths of the AIM reward and Q -function. It selects goals based on a weighted sum of the negative AIM proximity (favoring accessible goals) and the Q -value (favoring strategically valuable goals). By incorporating both, this approach aims for robustness across different levels of complexity of the environment. As shown in Figure 18, this combined strategy often achieves a performance between AIM reward and Q -function (Figures 8d, 9d and 10d). It represents a practical compromise, leveraging both learned proximity and value estimation for curriculum goal selection.

5.2 Comparison with State-of-the-Art methods

The results, illustrated in Figure 3 and detailed in Table 2, highlight PALDIFFCURL’s effectiveness. For the sake of comparison with the baseline methods, we utilized for PALDIFFCURL the best-performing goal selection strategy for each respective environment: namely, Averaging for PointUMaze and PointNMaze, and the Q -function for PointSpiralMaze. Figure 3 shows that PALDIFFCURL consistently achieves higher success rates more rapidly than the ten baseline methods across all three maze environments. Table 2 further quantifies this, showing that PALDIFFCURL reaches a 1.0 success rate significantly faster, an advantage that is more pronounced in complex environments. Notably, strong baselines like HGG and GRADIENT, while competitive in PointUMaze, often fail to achieve full success in the more challenging mazes within the given timeframe, underscoring PALDIFFCURL’s robustness and efficiency.

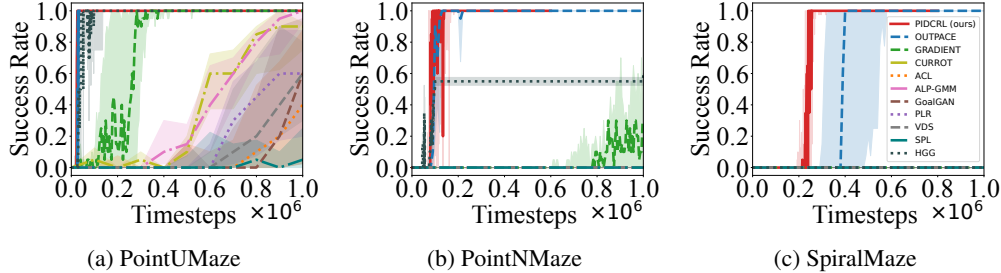


Figure 3: Comparison of test success rates for PALDIFFCURL and baseline CRL algorithms across three maze environments. Curves represent the mean success rate over five random seeds, with shaded areas indicating the standard deviation.

Table 2: No. of timesteps (rounded) to reach success rate 1.0 (average $\pm$ std. dev. across 5 runs). Only methods reaching a success rate 1.0 in all runs within $1e6$ timesteps per environment are included; methods failing in any run (seed) for a given environment are excluded.

Algorithm	PointUMaze	PointNMaze	PointSpiralMaze
PALDIFFCURL (Ours)	25400 $\pm$ 1020	87625 $\pm$ 21702	234125 $\pm$ 23089
OUTPACE	29800 $\pm$ 4166	113333 $\pm$ 24267	396875 $\pm$ 111451
HGG	48750 $\pm$ 24314	-	-
GRADIENT	263431 $\pm$ 114795	-	-

6 Conclusion and Limitations

We introduced PALDIFFCURL, a novel CRL approach that utilizes the Palette diffusion model [12] for curriculum goal generation. The method first trains the diffusion model on a generated dataset comprising agent trajectory heatmaps and associated curriculum goals (in our experiments, generated via OUTPACE [4]). By framing curriculum goal generation as an image inpainting task using masked heatmaps (Section 4.2), the model learns to predict suitable goals based on trajectory patterns. During subsequent RL training with PALDIFFCURL (Section 4.3), the agent’s current trajectory heatmap (without explicit goal information) is masked and fed as input to the trained diffusion model. The model then generates multiple curriculum goal candidates by completing the masked image. An optimal goal is chosen from these candidates using a selection strategy (Section 4.4) to guide the agent in the next training episode. Our experiments demonstrate that PALDIFFCURL effectively generates challenging yet achievable goals, matching or surpassing the performance of existing CRL methods in three complex maze environments. The method could be applicable, for instance, in achieving autonomous navigation in unknown environments by leveraging existing trajectories, e.g., collected from preliminary explorations. Possible knowledge transfer (in the form of trajectories) across different domains/environments could also be envisioned. In this regard, the creation of ad-hoc trajectory datasets from different environments could provide a valuable resource for practical CRL.

Key limitations include the reliance on an initially generated dataset for diffusion model training and the computational cost associated with the diffusion model’s inference step during RL training. Additionally, the current trajectory representation is inherently 2D, potentially requiring adaptation for

higher-dimensional state spaces. Future research should investigate methods for generating training data without synthetic supervision, explore more computationally efficient diffusion architectures, and extend the trajectory representation to handle 3D or more complex environments.

References

- [1] Wong, Esther and Leung, Kin and Field, Tony. State-space decomposition for reinforcement learning. Technical report, Department of Computing, Imperial College London, London, UK, 2021.
- [2] Jiang, Minqi and Grefenstette, Edward and Rocktäschel, Tim. Prioritized Level Replay. In *International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 4940–4950. PMLR, 2021.
- [3] Li, Kevin and Gupta, Abhishek and Reddy, Ashwin and Pong, Vitchyr H and Zhou, Aurick and Yu, Justin and Levine, Sergey. MURAL: Meta-Learning Uncertainty-Aware Rewards for Outcome-Driven Reinforcement Learning. In *International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 6346–6356. PMLR, 2021.
- [4] Daesol Cho and Seungjae Lee and H. Jin Kim. Outcome-directed Reinforcement Learning by Uncertainty & Temporal Distance-Aware Curriculum Goal Generation. In *International Conference on Learning Representations*, 2023.
- [5] Pascal Klink and Hany Abdulsamad and Boris Belousov and Carlo D’Eramo and Jan Peters and Joni Pajarinen. A Probabilistic Interpretation of Self-Paced Learning with Applications to Reinforcement Learning. *Journal of Machine Learning Research*, 22(182):1–52, 2021.
- [6] Klink, Pascal and D’Eramo, Carlo and Peters, Jan R and Pajarinen, Joni. Self-Paced Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 9216–9227. Curran Associates, Inc., 2020.
- [7] Klink, Pascal and Abdulsamad, Hany and Belousov, Boris and Peters, Jan. Self-Paced Contextual Reinforcement Learning. In *Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 513–529. PMLR, 2020.
- [8] Portelas, Rémy and Colas, Cédric and Hofmann, Katja and Oudeyer, Pierre-Yves. Teacher algorithms for curriculum learning of Deep RL in continuously parameterized environments. In *Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 835–853. PMLR, 2020.
- [9] Ren, Zhizhou and Dong, Kefan and Zhou, Yuan and Liu, Qiang and Peng, Jian. Exploration via Hindsight Goal Generation. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [10] Fang, Meng and Zhou, Tianyi and Du, Yali and Han, Lei and Zhang, Zhengyou. Curriculum-guided Hindsight Experience Replay. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [11] Florensa, Carlos and Held, David and Geng, Xinyang and Abbeel, Pieter. Automatic goal generation for reinforcement learning agents. In *International Conference on Machine Learning*, pages 1515–1528. PMLR, 2018.
- [12] Saharia, Chitwan and Chan, William and Chang, Huiwen and Lee, Chris and Ho, Jonathan and Salimans, Tim and Fleet, David and Norouzi, Mohammad. Palette: Image-to-image diffusion models. In *ACM SIGGRAPH 2022 Conference Proceedings*, pages 1–10, 2022.
- [13] Narvekar, Sanmit and Peng, Bei and Leonetti, Matteo and Sinapov, Jivko and Taylor, Matthew E and Stone, Peter. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020.
- [14] Sayar, Erdi and Iacca, Giovanni and Knoll, Alois. Multi-Objective Evolutionary Hindsight Experience Replay for Robot Manipulation Tasks. In *Genetic and Evolutionary Computation Conference*, page 403–411, New York, NY, USA, 2024. Association for Computing Machinery.

- [15] Klink, Pascal and Yang, Haoyi and D’Eramo, Carlo and Peters, Jan and Pajarinen, Joni. Curriculum Reinforcement Learning via Constrained Optimal Transport. In *International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 11341–11358. PMLR, 2022.
- [16] Huang, Peide and Xu, Mengdi and Zhu, Jiacheng and Shi, Laixi and Fang, Fei and ZHAO, DING. Curriculum Reinforcement Learning using Optimal Transport via Gradual Domain Adaptation. In *Advances in Neural Information Processing Systems*, volume 35, pages 10656–10670. Curran Associates, Inc., 2022.
- [17] Goodfellow, Ian and Pouget-Abadie, Jean and Mirza, Mehdi and Xu, Bing and Warde-Farley, David and Ozair, Sherjil and Courville, Aaron and Bengio, Yoshua. Generative Adversarial Nets. In *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014.
- [18] Schaul, Tom and Horgan, Daniel and Gregor, Karol and Silver, David. Universal Value Function Approximators. In *International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1312–1320. PMLR, 2015.
- [19] Zhang, Yunzhi and Abbeel, Pieter and Pinto, Lerrel. Automatic Curriculum Learning through Value Disagreement. In *Advances in Neural Information Processing Systems*, volume 33, pages 7648–7659. Curran Associates, Inc., 2020.
- [20] Alex Graves and Marc G. Bellemare and Jacob Menick and Rémi Munos and Koray Kavukcuoglu. Automated Curriculum Learning for Neural Networks. In *International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1311–1320. PMLR, 2017.
- [21] Durugkar, Ishan and Tec, Mauricio and Niekum, Scott and Stone, Peter. Adversarial Intrinsic Motivation for Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 34, pages 8622–8636. Curran Associates, Inc., 2021.
- [22] Chelsea Finn and Pieter Abbeel and Sergey Levine. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In *International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1126–1135. PMLR, 2017.
- [23] Haarnoja, Tuomas and Zhou, Aurick and Abbeel, Pieter and Levine, Sergey. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 2018.
- [24] Ho, Jonathan and Jain, Ajay and Abbeel, Pieter. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851. Curran Associates, Inc., 2020.
- [25] Blei, David M. and Kucukelbir, Alp and McAuliffe, Jon D. Variational Inference: A Review for Statisticians. *Journal of the American Statistical Association*, 112(518):859–877, 2017.
- [26] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.
- [27] Schulman, John and Wolski, Filip and Dhariwal, Prafulla and Radford, Alec and Klimov, Oleg. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.

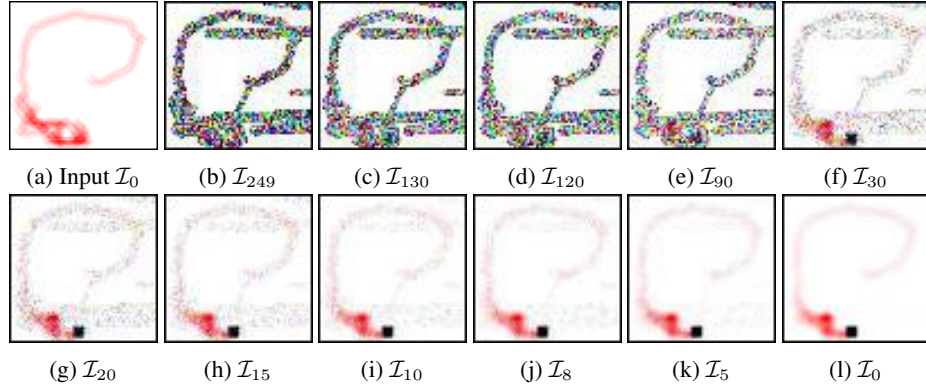


Figure 4: Examples of the output of the reverse project of the diffusion model input. The model takes as input the image (a). Note that the input image contains no goal information. A mask is then applied to the input image to mask the heatmap region, and the diffusion model incrementally denoises the masked region. The final output (l) generates a goal, which is then utilized as a curriculum goal for the next episode.

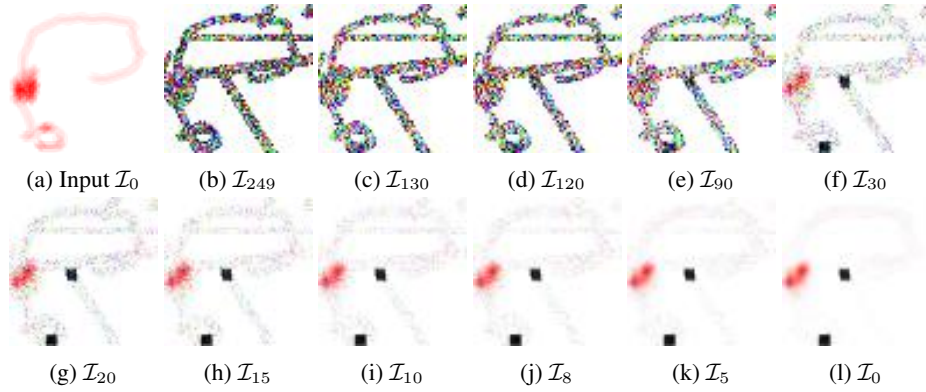


Figure 5: Example of a failure case during curriculum goal generation using the diffusion model. The reverse diffusion process, starting from the input heatmap (a), results in an output image (l) containing multiple potential goal locations (black squares) instead of a single, unambiguous goal.

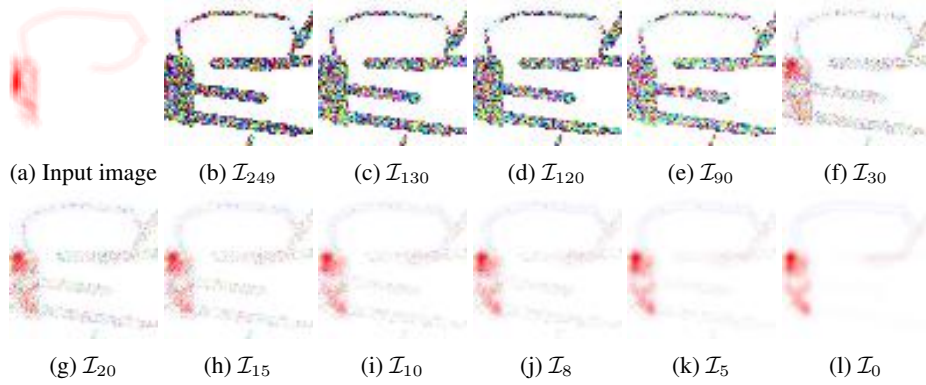


Figure 6: Example of a diffusion model failure during curriculum goal generation. Despite the iterative denoising process shown from (a) to (l), the model fails to synthesize a goal location (black square) in the final output image (l).

435 **B Goal generation and goal selection pseudocodes, and overall scheme of the**
 436 **proposed method**

Algorithm 2 Palette Goal Generation

```

1: Input: Trajectory heatmap (without goal)  $\mathcal{I}_0$ , no. of masks  $K$ , no. of reverse diffusion steps  $N$ 
2: for  $i = 0 \dots K$  do
3:   Generate path-based mask  $\mathcal{M}_i$ 
4:    $\eta \sim \mathcal{N}(0, I)$ 
5:    $\mathcal{I}_N = \mathcal{I}_0 \odot (1 - \mathcal{M}_i) + \eta \odot \mathcal{M}_i$  ▷  $\mathcal{I}_N$  should only be noisy in the masked area
6:    $\mathcal{G}_c \leftarrow \emptyset$ 
7:   for  $n = N \dots 0$  do ▷ Reverse diffusion steps
8:      $z \sim \mathcal{N}(0, I)$  if  $n > 1$  else  $z = 0$ 
9:      $\mathcal{I}_{n-1} = \frac{1}{\sqrt{\alpha_n}} \left( \mathcal{I}_n - \frac{1-\alpha_n}{\sqrt{1-\alpha_n}} f_\theta(x_i, y_n, \bar{\alpha}_n) \right) + \sqrt{1-\alpha_n} z$ 
10:     $\mathcal{I}_{n-1} = \mathcal{I}_0 \odot (1 - \mathcal{M}_i) + \mathcal{I}_{n-1} \odot \mathcal{M}_i$  ▷ Denoise only the masked area
11:     $g_c \leftarrow \text{PixelToCoord}(\mathcal{I}_0)$  ▷ Convert  $\mathcal{I}_0$  to grayscale and map to coordinates
12:    Add  $g_c$  to  $\mathcal{G}_c$ 
13: return  $\mathcal{G}_c$ 

```

Algorithm 3 Goal Selection Mechanism with Averaging

```

1: Input: generated curriculum goal candidates  $\mathcal{G}_c$ 
2:  $g_c = \frac{\sum_{i=1}^{|\mathcal{G}_c|} g_i}{|\mathcal{G}_c|}$ 
3: return  $g_c$  ▷ Return the selected goal

```

Algorithm 4 Goal Selection Mechanism with AIM reward

```

1: Input: Initial state  $s_0$ , generated curriculum goal candidates  $\mathcal{G}_c$ 
2:  $R \leftarrow \emptyset$ 
3: for each  $g_c \in \mathcal{G}_c$  do ▷ AIM reward function Eq. (4)
4:    $r_c = -r_\varphi(s_0, g_c)$ 
5:   Add  $r_c$  to  $R$ 
6: Compute probabilities  $p_i = \frac{\exp(R[i])}{\sum_{j=1}^{|\mathcal{G}_c|} \exp(R[j])}$ ,  $\forall i \in \{1, \dots, |\mathcal{G}_c|\}$ 
7: Sample  $i^* \sim \text{Categorical}(p_1, p_2, \dots, p_{|\mathcal{G}_c|})$  ▷ Select the index of the optimal goal
8:  $g_c = \mathcal{G}_c[i^*]$  ▷ Assign the goal corresponding to the sampled index
9: return  $g_c$  ▷ Return the selected goal

```

Algorithm 5 Goal Selection Mechanism with Q -function

```

1: Input: Sequential states  $s_{0:T} : s_0, \dots, s_T$  and actions  $a_{0:T} : a_0, \dots, a_T$ , generated curriculum goal set  $\mathcal{G}_c$ 
2:  $Q \leftarrow \emptyset$ 
3: for each  $g_c \in \mathcal{G}_c$  do
4:    $Q_c = Q_\phi(s_{0:T}, a_{0:T}, g_c)$ 
5:   Add  $Q_c$  to  $Q$ 
6: Compute probabilities  $p_i = \frac{\exp(Q[i])}{\sum_{j=1}^{|\mathcal{G}_c|} \exp(Q[j])}$ ,  $\forall i \in \{1, \dots, |\mathcal{G}_c|\}$ 
7: Sample  $i^* \sim \text{Categorical}(p_1, p_2, \dots, p_{|\mathcal{G}_c|})$  ▷ Select the index of the optimal goal
8:  $g_c = \mathcal{G}_c[i^*]$  ▷ Assign the goal corresponding to the sampled index
9: return  $g_c$  ▷ Return the selected goal

```

Algorithm 6 Goal Selection Mechanism with AIM reward & Q -function

- 1: **Input:** Sequential states $s_{0:T} : s_0, \dots, s_T$ and actions $a_{0:T} : a_0, \dots, a_T$, generated curriculum goal set $\mathcal{G}_c$
 - 2: $Z \leftarrow \emptyset$
 - 3: **for** each $g_c \in \mathcal{G}_c$ **do**
 - 4: $Q_c = Q_\phi(s_{0:T}, a_{0:T}, g_c)$
 - 5: $r_c = -r_\varphi(s_0, g_c)$
 - 6: Add $(\xi_r \cdot r_c + \xi_q \cdot Q_c)$ to Z
 - 7: Compute probabilities $p_i = \frac{\exp(Z[i])}{\sum_{j=1}^{|\mathcal{G}_c|} \exp(Z[j])}$, $\forall i \in \{1, \dots, |\mathcal{G}_c|\}$
 - 8: Sample $i^* \sim \text{Categorical}(p_1, p_2, \dots, p_{|\mathcal{G}_c|})$ ▷ Select the index of the optimal goal
 - 9: $g_c = \mathcal{G}_c[i^*]$ ▷ Assign the goal corresponding to the sampled index
 - 10: **return** g_c ▷ Return the selected goal
-

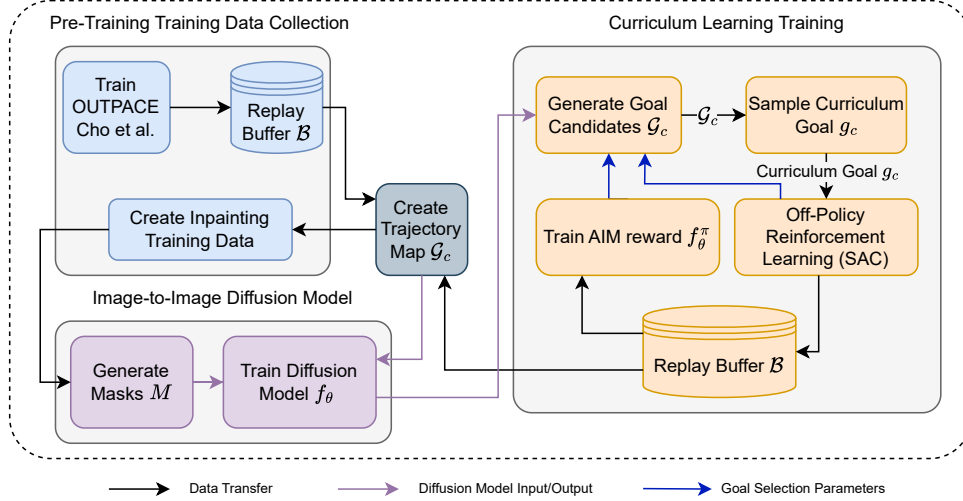


Figure 7: Training pipeline adopted in this work.

437 The overall training pipeline is illustrated in Figure 7. Initially, a trajectory dataset, comprising state
 438 sequences and corresponding curriculum goals, is generated by training a baseline agent with the
 439 OUTPACE method [4] independently for each maze environment. This data is then transformed into
 440 an image dataset; each sample is a trajectory heatmap visualizing the agent’s path and marked with the
 441 episode’s curriculum goal. A masking mechanism (Section 4.2) is applied to each heatmap, typically
 442 occluding portions of the trajectory and the goal location, thereby framing the task as an image
 443 inpainting problem. A diffusion model is subsequently trained on this masked dataset to reconstruct
 444 the occluded regions, learning the relationship between trajectory heatmaps and corresponding
 445 curriculum goals. Once trained, this diffusion model is employed during the PALDIFFCURL training
 446 phase (Section 4.3). In each training episode, the agent’s trajectory is converted into a heatmap
 447 image, which excludes explicit goal information. Given a heatmap, k distinct binary masks are
 448 applied to generate a batch of k partially masked inputs. These masked heatmaps serve as input to the
 449 trained diffusion model, which generates candidate curriculum goals for the next episode based on
 450 the observed trajectory pattern. Finally, an optimal curriculum goal is selected from the candidates
 451 using the mechanism described in Section 4.4 and utilized in the PALDIFFCURL training process.

452 C Generated Curriculum Goals

453 This section presents visualizations of the curriculum goals generated by PALDIFFCURL using the
 454 different goal selection strategies discussed in Section 4.4. The figures illustrate the types of goals
 455 produced in the PointUMaze, PointNMaze, and PointSpiralMaze environments. Each subfigure
 456 showcases the goals generated by a specific selection strategy: Averaging, AIM reward, Q -function,
 457 and the combined AIM reward & Q -function approach. These visualizations provide qualitative
 458 insight into how each strategy influences goal placement within the respective maze structures.

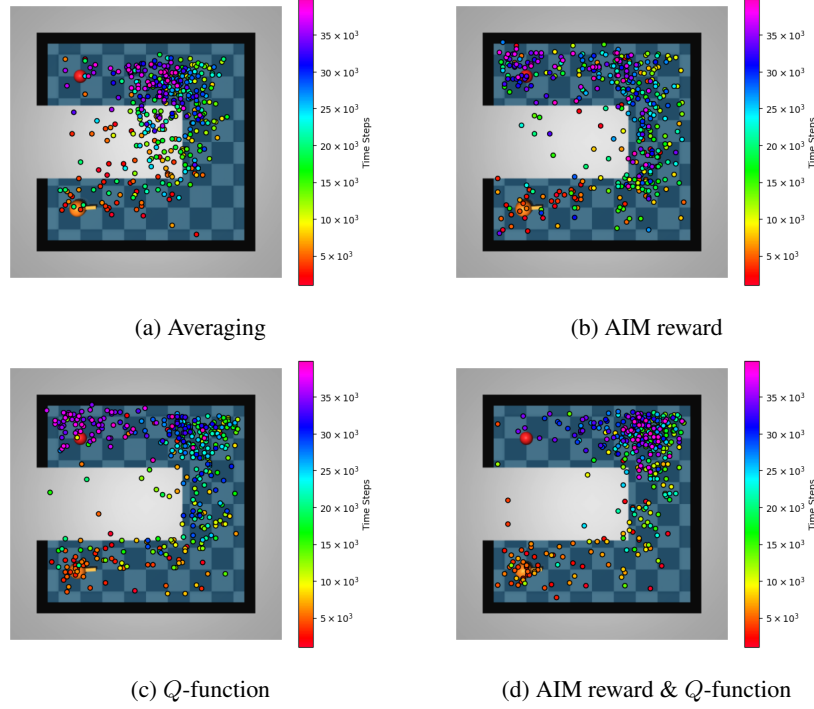


Figure 8: PointUMaze curriculum goals.

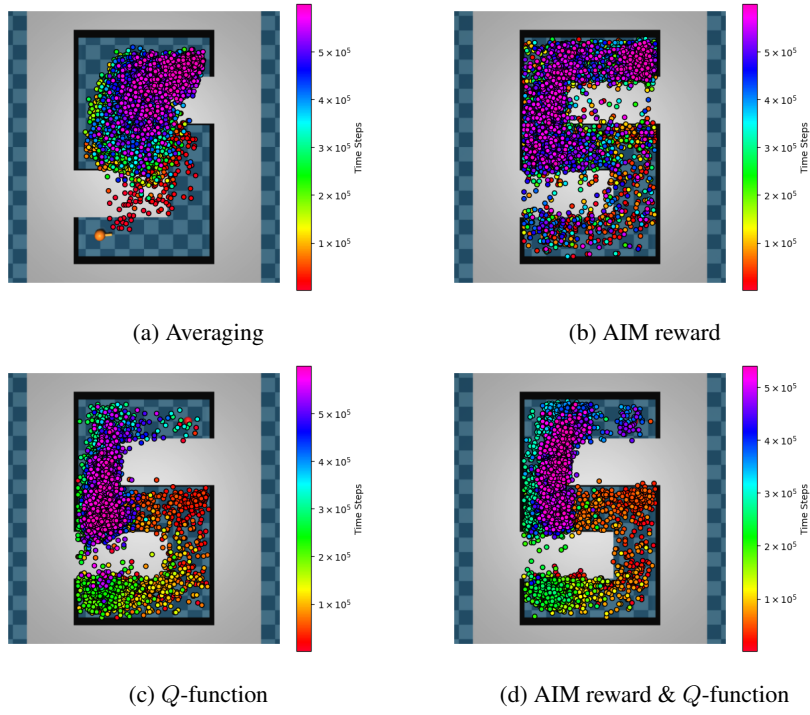


Figure 9: PointNMaze curriculum goals.

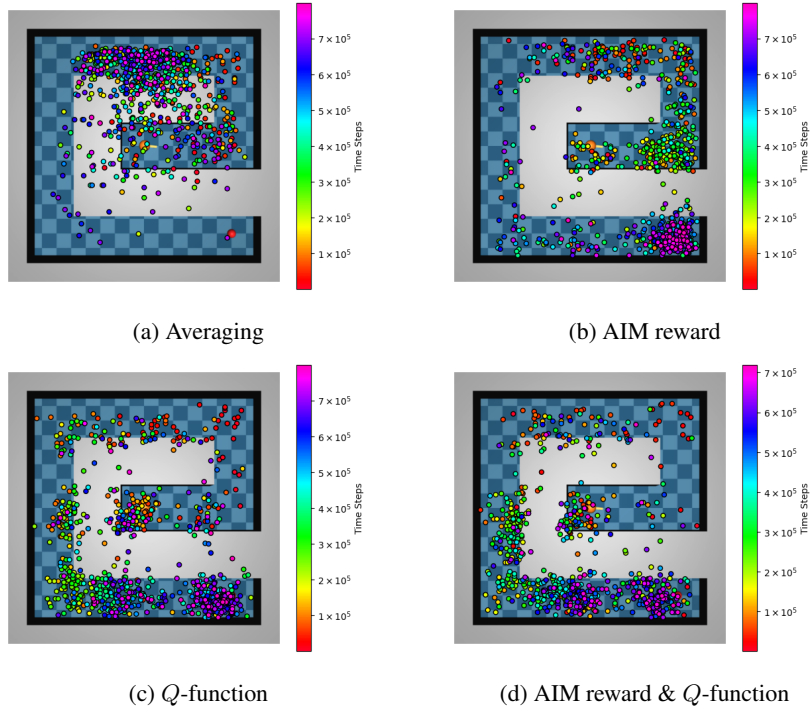


Figure 10: PointSpiralMaze curriculum goals.

D Experimental Details

The implementation of PALDIFFCURL was based on the original OUTPACE codebase [4], augmented with the Palette diffusion model for curriculum goal generation. Experiments were performed on a workstation equipped with a 4-core CPU, an NVIDIA RTX A5000 GPU, and 64GB of RAM.

Baselines. The baseline CRL algorithms were trained using their respective original implementations and default settings, sourced as follows:

- HGG [9]: Default settings from the original implementation at <https://github.com/Stilwell-Git/Hindsight-Goal-Generation>.
- CURROT [15], ALP-GMM [8], VDS [19], SPRL [5], PLR [2], ACL [20], GoalGAN [11]: Default settings from the implementation available at <https://github.com/psclklnk/currot>.
- GRADIENT [16]: Default settings from the implementation available at <https://github.com/PeideHuang/gradient>.
- OUTPACE [4]: Default settings from the implementation available at https://github.com/jayLEE0301/outpace_official.

Training details. All baseline models, except for GRADIENT, were trained using SAC [23]. Although the original implementations of these algorithms primarily utilized on-policy methods, they were adapted in the CURROT repository to employ the off-policy SAC algorithm. This modification facilitates a direct comparison of sample efficiency across the models. The GRADIENT method was trained using both SAC and Proximal Policy Optimization (PPO) [27]; however, we report only the results obtained with PPO, as it demonstrated superior performance compared to SAC in the three considered maze environments. The hyperparameters used for PALDIFFCURL during the experiments are detailed in Table 3.

Environment details. For each task, the agent’s state vector s includes its planar position, linear velocity components (v_x, v_y) , orientation angle, and angular velocity about the z -axis. The agent’s actions control its linear and angular velocities. Each episode begins with the agent positioned at the origin $[0, 0]$. The target goal distribution is generated by adding uniform noise to a nominal goal position. The nominal goal positions and map dimensions for the PointUMaze, PointNMaze, and PointSpiralMaze tasks are $[0, 8]$ within a 12×12 map, $[8, 16]$ within a 12×20 map, and $[8, -8]$ within a 20×20 map, respectively. An episode is deemed successful if the agent reaches the sampled target goal within a distance threshold of 0.5 units. The `CoordtoPixel` function, referenced in line 10 of Algorithm 1, maps the environment dimensions to pixel space using the `image_scale` factor. The agent’s trajectory is then rendered in this pixel space, with its visual size determined by the `agent_scale` factor. Both scale factors are detailed in Table 4.

For every observed agent position during the episode, the coordinates of that position were scaled by the `image_scale` and then mapped onto the image. To ensure that the mapped trajectory is continuous, the area from the previous position to the current one is also mapped based on the `agent_scale`. After mapping the whole trajectory for the episode, the goal is inserted into the image with a size that is the same as the agent’s size. The exact scaling values for every environment are detailed in Table 4.

Table 3: Hyperparameters for PALDIFFCURL.

Hyperparameter	Value	Hyperparameter	Value
Input image size	64x64	Initialization	Kaiming
Training batch size	8	Learning optimizer	Adam
Validation batch size	8	U-Net input channel	6
Mask Type	Path-based	U-Net output channel	3
# of epochs	300	U-Net depth multipliers	1, 2, 4, 8
# of iters per episode	2000	U-Net attention resolution	8
Loss	MSE	U-Net dropout rate	0.2
Evaluation Metric	MAE	U-Net residual blocks	2
EMA scheduler start	1	Diffusion noise schedule	Linear
EMA scheduler decay	0.999	Diffusion noise interval	[1e-6, 1e-2)
Learning rate	5e-5	Train denoising steps	2000
Weight decay	0	Infer denoising steps	250

Table 4: Hyperparameters for data collection.

Environment	Image Scale	Agent Scale
PointUMaze	25	25
PointNMaze	25	25
PointSpiralMaze	25	38

497 E Cumulative state visitation heatmaps

498 This section provides visualizations of the cumulative state visitation heatmaps for the different
 499 goal selection strategies discussed in Section 4.4. These heatmaps illustrate the agent’s exploration
 500 patterns over time in the PointUMaze, PointNMaze, and PointSpiralMaze environments. Figures
 501 11-13) show how the agent’s visitation frequency, represented by color intensity (darker shades
 502 indicate less frequent visits), evolves across the maze at different stages of training (timestep t). By
 503 comparing the heatmaps generated by the Averaging, AIM reward, Q -function, and the combined
 504 AIM reward & Q -function strategies, we can gain insights into how each approach influences the
 505 agent’s exploratory behavior and its ability to cover the state space effectively.

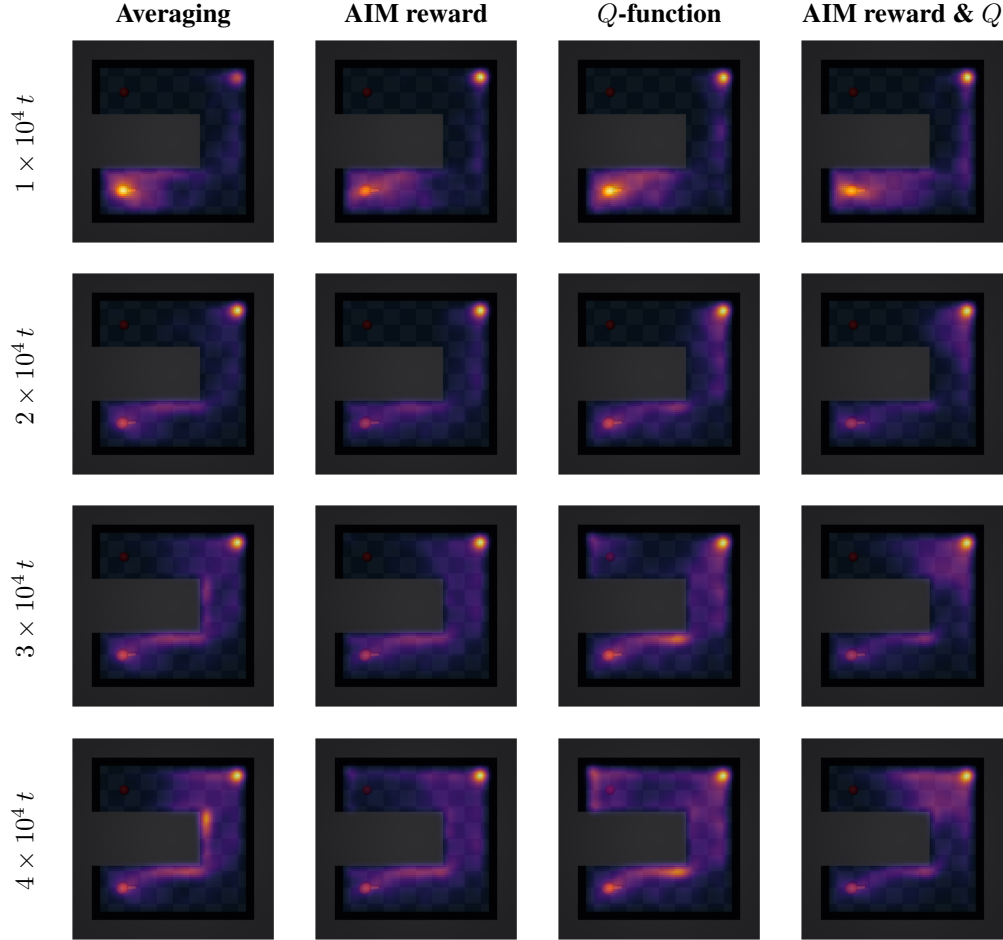


Figure 11: Cumulative state visitation heatmaps in PointUMaze for different goal selection strategies (PALDIFFCURL variants). Each row shows the heatmap accumulated up to the indicated training timestep (t).

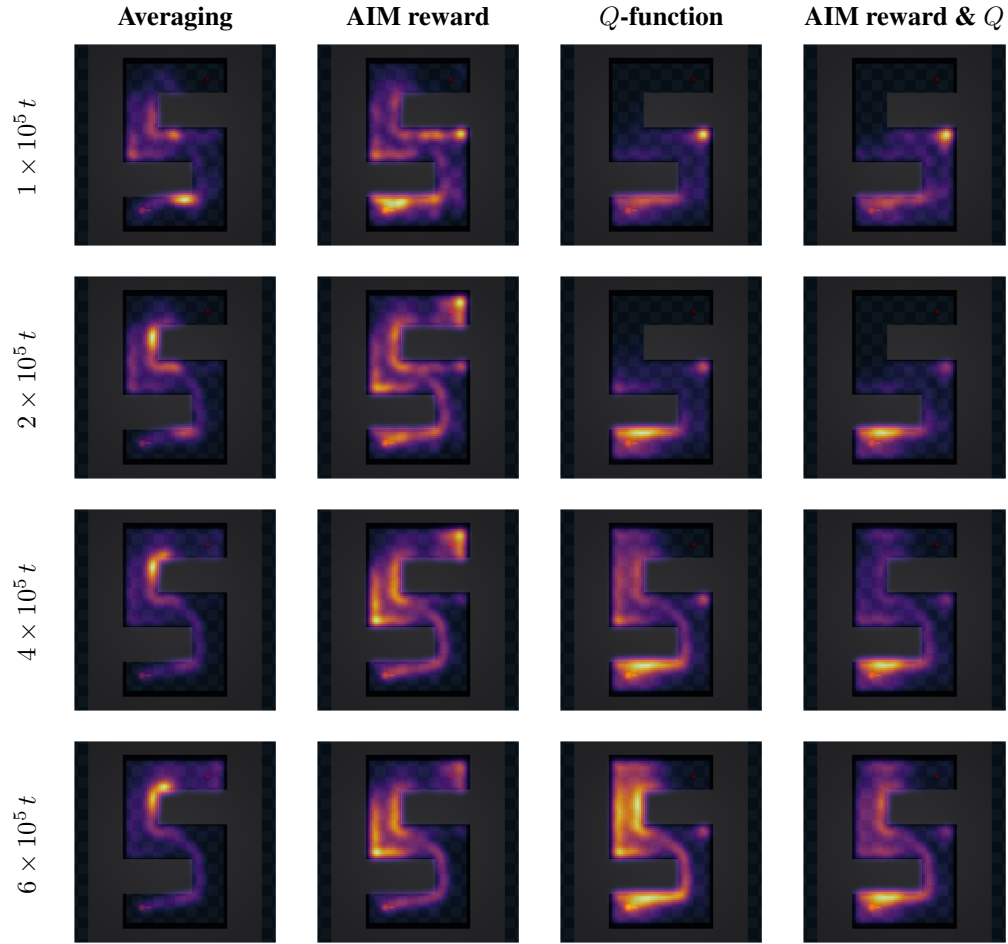


Figure 12: Cumulative state visitation heatmaps in PointNMaze for different goal selection strategies (PALDIFFCURL variants). Each row shows the heatmap accumulated up to the indicated training timestep (t).

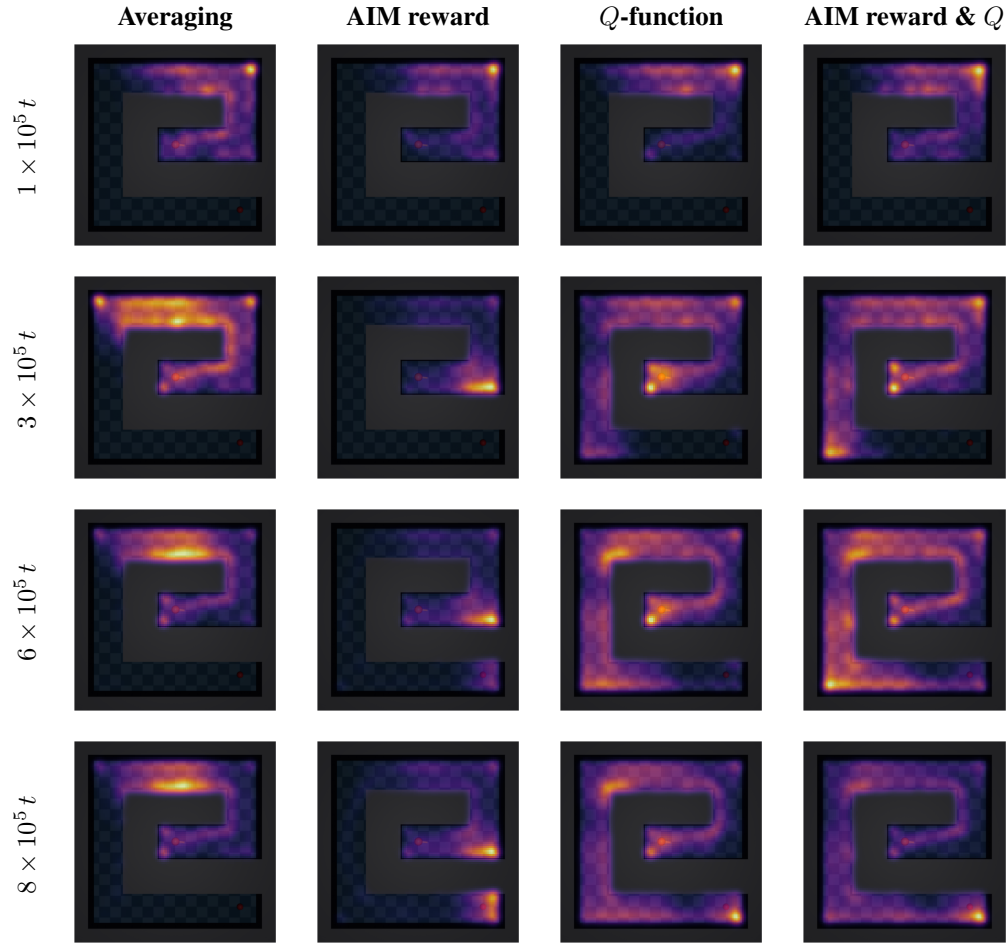


Figure 13: Cumulative state visitation heatmaps in PointSpiralMaze for different goal selection strategies (PALDIFFCURL variants). Each row shows the heatmap accumulated up to the indicated training timestep (t).

F Visualization of rewards and proximity

In our framework, we utilize a single learned scalar function, denoted as $r_\varphi^\pi(s, g)$ (derived from the AIM framework, see Section 3.3), which inherently captures the relationship between states and goals. Empirically, the output of this function $r_\varphi^\pi(s, g)$ increases as the state s gets closer to the goal g in the environment space. We leverage this function in two distinct but related ways:

1. **Reward View (Fixed Goal):** When training the RL agent’s policy, we fix the target goal g for the episode. In this context, we interpret $r_\varphi(s, g)$, as the reward signal. The agent’s objective is to learn a policy $\pi(a|s, g)$ that *maximizes* this reward $r_\varphi(s, g)$, effectively driving the agent towards the fixed goal g . This aligns with the standard RL paradigm of maximizing cumulative rewards.
2. **Proximity View (Fixed State):** When selecting the next curriculum goal g_c using the AIM-based strategy (Section 4.4, Algorithm 4), we fix the agent’s initial state s_0 for the episode. Here, we directly use the output of the learned function, $r_\varphi^\pi(s_0, g)$, as a measure of proximity or cost. Since lower values indicate that a potential goal g is closer or more accessible from s_0 , the curriculum goal selector aims to *minimize* this value among the generated candidates $\mathcal{G}_c$:

$$g_c = \arg \min_{g \in \mathcal{G}_c} r_\varphi^\pi(s_0, g). \quad (13)$$

This selects curriculum goals that are progressively getting closer to the desired goal.

It is crucial to note that the “Reward View” and the “Proximity View” utilize the same underlying learned function f_φ^π . The difference lies in which argument is fixed and whether we maximize its negative (Reward View) or minimize its direct output (Proximity View). This dual interpretation allows us to use the learned relationship between states and goals consistently for both policy optimization and curriculum generation. Visualizations comparing these two views for the PointUMaze, PointNMaze, and PointSpiralMaze environments are shown in Figures 14, 15, and 16, respectively.

Table 5: Wilcoxon rank-sum applied to the success rates from 5 independent runs of PALDIFFCURL (ours) and OUTPACE on the three maze environments.

Environment	statistic	p-value
PointUMaze	-2.04	0.021
PointNMaze	2.32	0.990
PointSpiralMaze	-3.88	0.001

Table 6: Computation time analysis comparing the execution time of PALDIFFCURL (whole algorithm), only the diffusion model in PALDIFFCURL, and OUTPACE. It can be noted that the diffusion model being responsible for 84.85% of the whole training time of PALDIFFCURL. During these measurements, all methods have been trained for 40000 timesteps in PointUMaze environment.

Method	Time	Percentage
PALDIFFCURL (whole algorithm)	2685.54 seconds	100.00%
Only diffusion model in PALDIFFCURL	2278.70 seconds	84.85%
OUTPACE	764.22 seconds	-

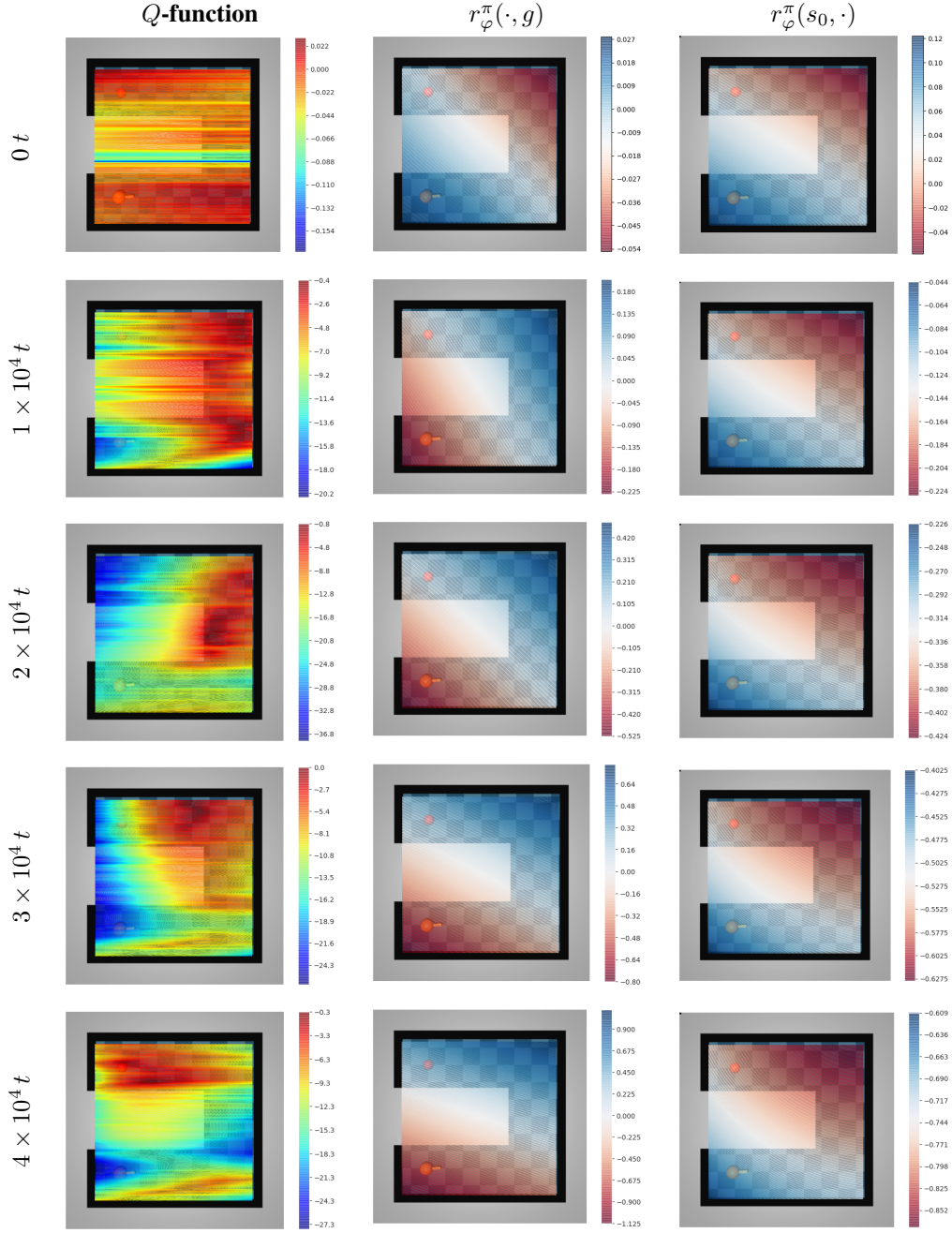


Figure 14: Visualization of Q and Aim Reward function across training in PointUMaze. Left column: Evolution of the Q -function with contour plots at 100 density levels. Middle column: Reward landscape $r_\varphi^\pi(s, g)$ for a fixed target goal g (contour plots at 100 density levels), which the policy is trained to maximize. Right column: Proximity landscape $r_\varphi^\pi(s_0, \cdot)$ for a fixed initial state s_0 (contour plots at 100 density levels), which is minimized during curriculum goal selection. Each row represents a snapshot at increasing training timesteps (t).

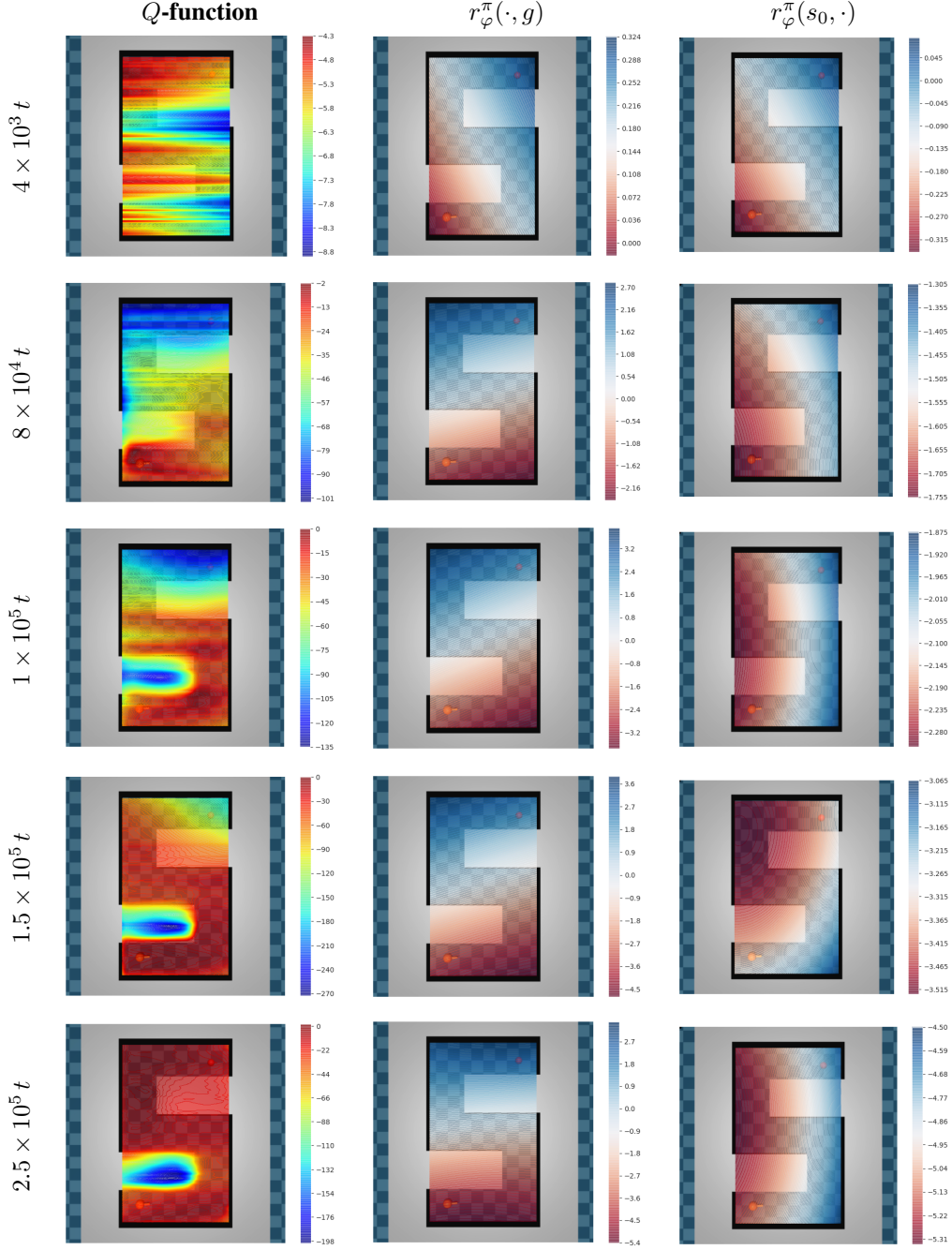


Figure 15: Visualization of Q and Aim Reward function across training in PointNMaze. Left column: Evolution of the Q -function with contour plots at 100 density levels. Middle column: Reward landscape $f_{\varphi}^{\pi}(s, g)$ for a fixed target goal g (contour plots at 100 density levels), which the policy is trained to maximize. Right column: Proximity landscape $r_{\varphi}^{\pi}(s_0, \cdot)$ for a fixed initial state s_0 (contour plots at 100 density levels), which is minimized during curriculum goal selection. Each row represents a snapshot at increasing training timesteps (t).

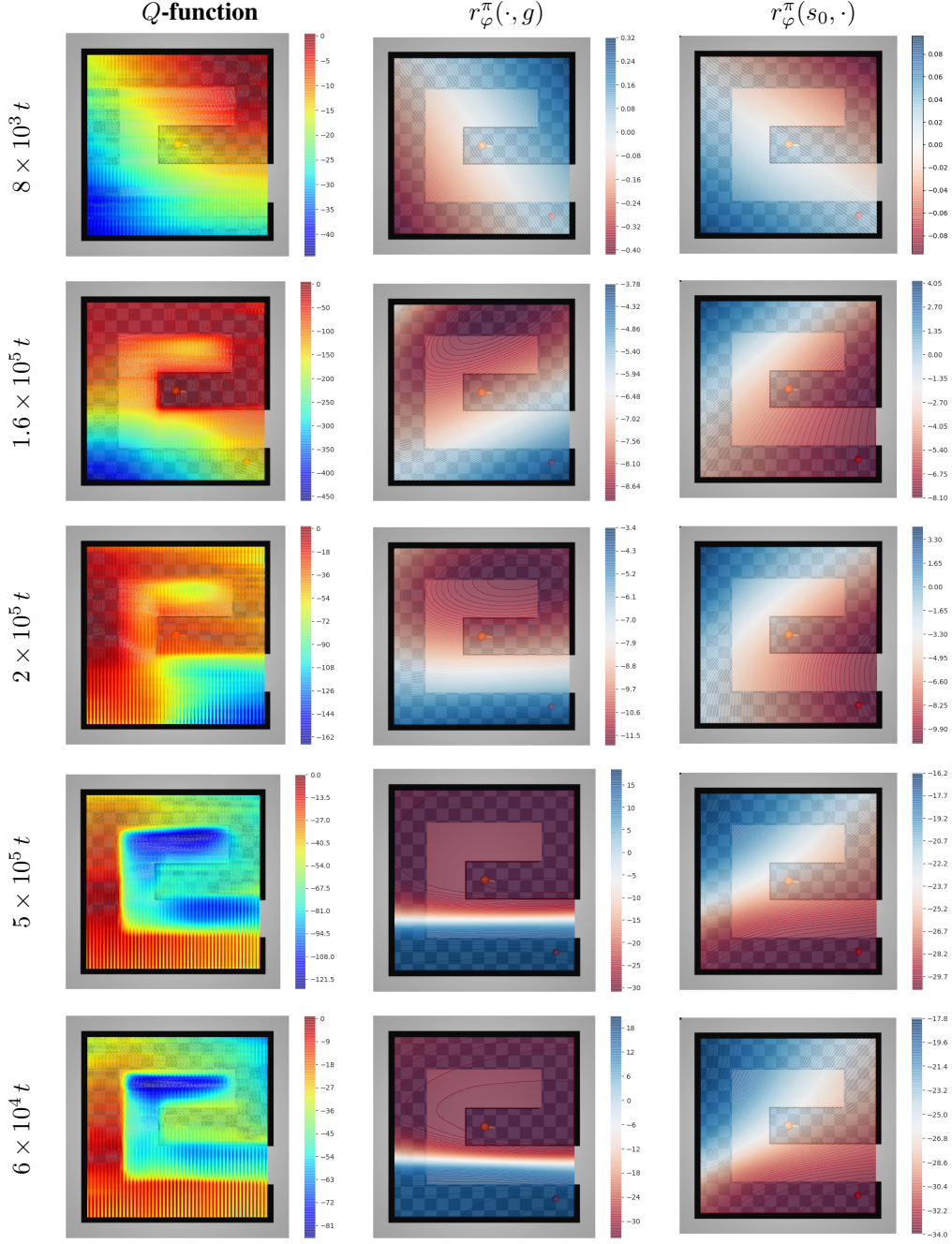


Figure 16: Visualization of Q and Aim Reward function across training in PointSpiralMaze. Left column: Evolution of the Q -function with contour plots at 100 density levels. Middle column: Reward landscape $r_\varphi^\pi(s, g)$ for a fixed target goal g (contour plots at 100 density levels), which the policy is trained to maximize. Right column: Proximity landscape $r_\varphi^\pi(s_0, \cdot)$ for a fixed initial state s_0 (contour plots at 100 density levels), which is minimized during curriculum goal selection. Each row represents a snapshot at increasing training timesteps (t).

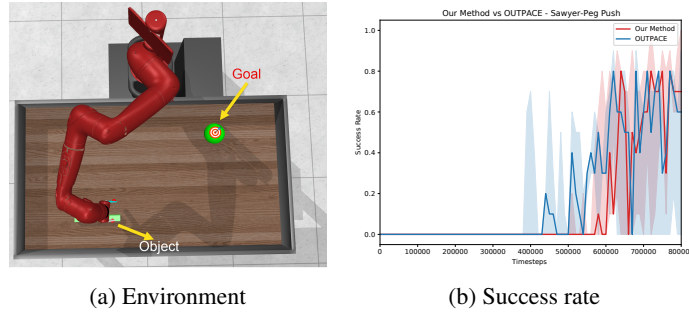


Figure 17: Comparison of test success rates for PALDIFFCURL and OUTPACE (baseline CRL algorithm) in the Sawyer Push environment (https://github.com/vikashplus/sawyer_sim). The curves represent the mean success rate over five random seeds, with shaded areas indicating the standard deviation.

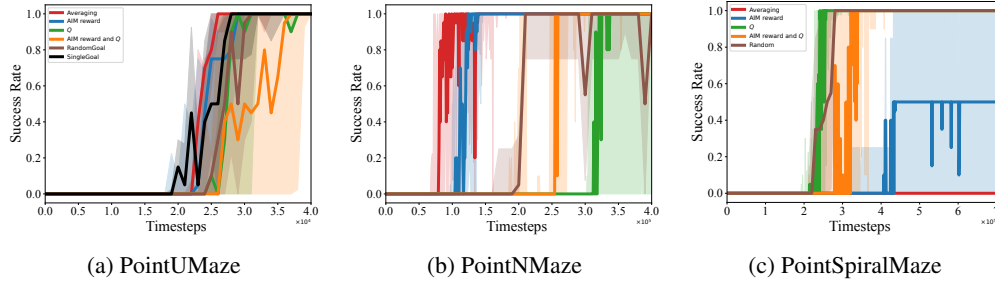


Figure 18: Performance of different goal selection strategies for PALDIFFCURL in the three maze tasks (note that, due to computational resource limitations, not all strategies have been tested on PointNMaze and PointSpiralMaze—these experiments will be completed after the rebuttal period).

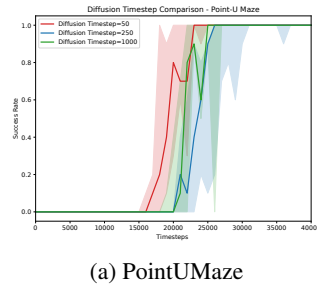


Figure 19: Ablation on the effect of the number of diffusion timesteps in the PointUMaze environment. The curves represent the mean success rate over five random seeds for PALDIFFCURL with varying timesteps (50, 250, and 1000), with shaded areas indicating the standard deviation.